

Comparative Analysis of Sequential Models for Mobile Network Traffic Forecasting

Emmanuel NSABAGASANI — African Leadership University
Formative Assignment 1, September 2026

Abstract

Short-horizon forecasts of cellular traffic underpin dynamic resource allocation and base-station energy saving, yet radio access networks still account for 73–87% of mobile-operator energy consumption [23], [24]. This study compares three sequential models — seasonal ARIMA, LSTM, and temporal convolutional network (TCN) — for one-step-ahead (10-minute) Internet activity forecasting on the Telecom Italia Milan grid [1], and asks how performance varies across areas with different traffic characteristics.

The 19.4 GB source dataset was reduced to a 341 MB memory-mapped `float32` store by streaming aggregation; peak resident memory fell from 408 to 149 MB (2.7×), in-memory payload from 296 to 11 MB (27×), never exceeding 241 MB against a 1 GB budget. Models were tuned on the highest-traffic area and applied unchanged to three areas, evaluated on 16–22 December 2013.

All models beat seasonal-naïve by a wide margin ($MASE \ll 1$), but seasonal naïve is weak here: lag-1 autocorrelation is 0.987 versus 0.878 at lag 144. Against persistence, SARIMA is 7.6–16.4% worse on every area. The TCN improved on persistence by 14.8%, 15.4%, and 15.5% — spread 0.007 across areas differing fivefold in level — while the LSTM achieved a similar mean (0.905 vs 0.848) with 25× larger spread. A controlled TCN receptive-field study found no measurable effect between 61 and 509 steps; levels = 4 was selected for parsimony. Errors concentrate at the daily peak; the worst window on the busiest area was the Friday-to-Saturday transition (+48% day-over-day), not an unprecedented pre-Christmas peak, and no model receives day-of-week input.

1. Introduction

Mobile operators allocate capacity against demand that varies by an order of magnitude within a day and between neighbouring city blocks. Short-horizon forecasts support dynamic spectrum allocation, load balancing, admission control, and energy-saving cell sleep modes [25] — decisions where under-provisioning degrades service at peak and over-provisioning wastes energy in a sector where RANs consume 73–87% of operator electricity [23], [24].

This study uses the Telecom Italia Milan dataset [1]: telecommunications activity on a 100×100 grid of ~ 235 m squares at 10-minute resolution for two months [2], [3]. At 19.4 GB raw and ~ 320 million rows, the dataset does not fit in working memory on the hardware used; data handling is therefore part of the research problem.

Research question. *How do different sequential models compare for one-step-ahead mobile network traffic forecasting, and how does performance vary across geographical areas with different traffic characteristics?*

Objectives.

1. Design a data pipeline tractable within ~ 1 GB working memory and quantify its trade-offs.
2. Characterise temporal and spatial structure and derive model configurations from it.
3. Implement SARIMA, LSTM, and TCN; tune on one area; evaluate unchanged on three areas.
4. Test whether neural complexity is justified against strong naïve baselines and identify failure modes.

The comparison uses identical splits, scaling, evaluation harness, and error metrics in original activity units.

2. Related Work

The Milan Big Data Challenge release [1] provides an *activity measure* — CDR events apportioned to grid squares by coverage overlap — not byte volumes. The 100×100 grid and Nov 2013–Jan 2014 window are standard in downstream work [2], [3]. Zhang, Patras, and Haddadi [9] survey deep learning in mobile networking; this study focuses on short-horizon single-cell forecasting.

Seasonal ARIMA with dual periodicities is established for network traffic [10]; here we difference at lag 144 (daily) only, not lag 1,008 (weekly), because a second seasonal difference would consume another week of a 62-day series. Stationarity is assessed with ADF [19] and KPSS [20] together [11]. Prior Milan studies often compare deep models to non-seasonal ARIMA [2] or aggregate to hourly [3]; we retain native 10-minute resolution and fit a seasonally differenced SARIMA on $\log_{10}$ scale as a fair linear baseline.

Deep models on this dataset include ConvLSTM/STCNet [4], spatial CNNs [3], graph models [6], and architecture comparisons [5]. Reported gains are typically *long-horizon*: Zhang and Patras [2] show margins growing with horizon and shrinking at short horizons; Trinh *et al.* [7] separately evaluate one-step performance and input-window length. Bai *et al.* [13] report TCNs outperforming LSTMs on diverse tasks; Zhang *et al.* [28] combine TCN with attention on Milan data but compare neural baselines only, without persistence or classical references.

Evaluation follows Hyndman and Koehler [16]: MASE scales errors against a naïve benchmark, though we use seasonal-naïve on the evaluation partition rather than in-sample one-step naïve (Section 5.2). Makridakis *et al.* [21], [22] and Wang *et al.* [6] show simple methods remain competitive; Zeng *et al.* [29] find a "Repeat" baseline beating Transformers on some benchmarks but losing on long-horizon seasonal tasks — the opposite regime from one-step forecasting on a series with lag-1 ACF 0.987. No prior Milan study reports persistence or seasonal-naïve for single-cell one-step Internet traffic [2], [3], [4], [6], [28], so we compute those baselines here. STL decomposition [17], [18] and Huber loss [26] with Adam [27] follow standard practice; asymmetric operational costs [8] are noted but not used, as symmetric metrics are required.

3. Dataset and Data Preparation

3.1 The data

Call Detail Records from Telecom Italia [1] cover Milan as a 100×100 grid (~ 235 m squares) [2], [3]. For each area, 10-minute interval, and country code, the release records SMS, call, and Internet activity from 1 November 2013 to 1 January 2014 — 62 days, 8,928 intervals. Values are a normalised activity proxy [1]; comparisons are between models on the same area. This study forecasts **Internet activity** at native 10-minute resolution.

3.2 Computational constraint

Work ran on an Intel Core i5-6300U (2 cores, 4 threads, 2.4 GHz), 7.9 GB RAM, Windows 10, no GPU. Raw data: 19.38 GB, 319,896,289 rows — roughly 2.5× total RAM. Full `pd.read_csv` loading fails outright; peak memory must depend on chunk size, not dataset size.

3.3 Memory management strategy

Five decisions, in order:

- 1. **Aggregate country codes during parsing** — sums (square, interval, country) to (square, interval), reducing 319.9M rows to 89.28M (3.6×).
- 2. **Column projection** — `usecols=[0, 1, 7]` for square, interval, internet only.
- 3. **Chunked streaming** — 1M-row chunks folded into a pre-allocated accumulator; peak memory independent of file size.
- 4. **Dense float32 memmap store** — grid is >99.9% complete daily; sparse format would be larger. Store: **341 MB**; one area's series touches ~35 KB resident via `numpy.memmap [30]`.
- 5. **Download, process, discard** — one raw day at a time; peak disk ≈ 721 MB vs 19.38 GB. Accumulation in `float64`, cast to `float32` once per day.

Measured effect

Benchmarks on one day file (308 MB, 4.8M rows), each in a **separate subprocess** (CPython does not return freed heap to the OS):

variant	rows	seconds	baseline_rss_mb	peak_rss_mb	peak_above_baseline_mb	in_memory_payload_mb	raw_file_mb	reduction_vs_naive_%
naive	4842625	9.79	75.3	408	332.7	295.6	307.9	0
typed	4842625	14.8	75.5	273.9	198.4	157	307.9	32.9
projected	4842625	7.15	75.3	165.2	89.9	73.9	307.9	59.5
streamed	4842625	7.31	75.6	148.9	73.2	11	307.9	63.5

Peak RSS falls 408 → 149 MB (2.7×); in-memory payload 295.6 → 11.0 MB (27×). RSS includes ~75 MB interpreter baseline. Chunked streaming fixes payload at chunk size regardless of file count; naive loading would need ~62 × 296 MB ≈ 18 GB resident.

Across 62 days, peak RSS averaged 166 MB, maximum **241 MB**:

quantity	value
Raw dataset on disk (62 tab-separated day files)	19.38 GB
Raw rows parsed (square x interval x country code)	319,896,289
Rows after summing out country code	89,280,000
Row-count reduction factor	3.6x
Compact store, float32 dense matrix	341 MB
Same store as float64	681 MB
Same store retaining all 5 activity types (float32)	1703 MB
Disk footprint reduction (raw -> store)	58x
Peak disk needed by the pipeline (store + 1 raw day)	721 MB
Resident memory to read one area's full series	35 KB

Trade-offs

Only Internet activity is retained (all five channels → 1.70 GB store). Country-code breakdown is irrecoverable. `float32` is not bit-exact but below publisher noise floor. Dense layout assumes a complete grid (verified for Milan). Parallel HTTP range requests (8 workers) raised download from ~0.17 MB/s to 4–5 MB/s.

3.4 Access note

Harvard Dataverse requires a guestbook POST with `?signed=true` for authenticated downloads since v6.10; signed URLs expire ~1 minute and are re-issued on demand.

3.5 Data quality

Checks before modelling: all 62 days present, no NaNs, 0.019% zeros, zero out-of-range rows; city profile min 05:00 / max 13:00 CET and weekend/weekday ratio 0.83; grid orientation verified via `milano-grid.geojson` (r = 1.000); All Saints' Day (1 Nov) total 82.5M vs ~110M on ordinary weekdays.

4. Exploratory Analysis

Exploratory analysis motivates modelling choices in Section 5.

4.1 Spatial distribution

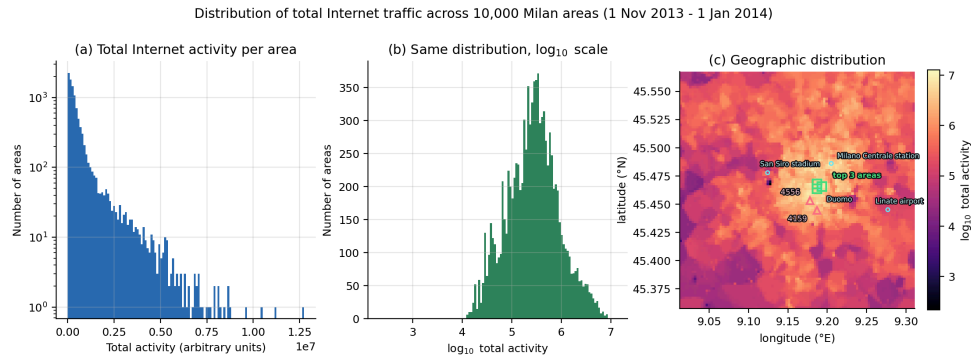


Figure 1. Total Internet activity per area. (a) Right-skewed distribution. (b) Log-scale unimodal shape. (c) Map with three highest-traffic areas (green) and reference areas 4159, 4556 (pink).

Statistic	Value
Mean / median total activity	555,289 / 277,871
Maximum + median	45.8
Gini coefficient	0.608
Top 1% / top 10% share	11.05% / 48.39%

Traffic concentrates in a historic-centre core (Duomo area); the three busiest squares are contiguous within 0.5 km. A single-area conclusion cannot generalise city-wide — motivating evaluation on three areas with unchanged hyperparameters.

4.2 Area time series

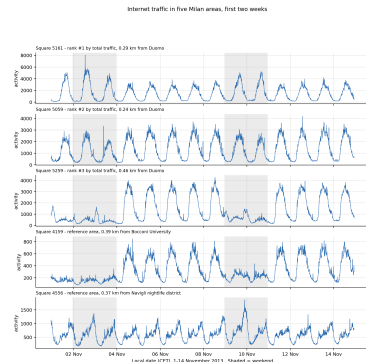


Figure 2. Internet traffic, 1–14 November 2013, for three highest-traffic and two reference areas. Weekends shaded.

square	role	total	mean	std	cv	min	max	peak_over_trough	weekday_mean	weekend_mean	weekend_ratio	peak_hour
5161	Square 5161 - rank #1 by total traffic, 0.29 km from Duomo	1.274e+07	1427	1382	0.968	55.795	8044	45.641	1283	1777	1.385	16
5059	Square 5059 - rank #2 by total traffic, 0.24 km from Duomo	1.117e+07	1251	961.472	0.768	108.987	4666	16.486	1304	1123	0.861	14
5259	Square 5259 - rank #3 by total traffic, 0.46 km from Duomo	1.049e+07	1174	1103	0.939	72.542	4442	9.264	1412	599.634	0.425	13
4159	Square 4159 - reference area, 0.39 km from Bocconi University	2.454e+06	274.881	181.329	0.66	47.264	940.891	7.425	312.877	183.345	0.586	12
4556	Square 4556 - reference area, 0.37 km from Navigli nightlife district	4.575e+06	512.396	248.267	0.485	80.247	1869	7.28	492.786	561.34	1.139	22

Five areas differ fivefold in level and in *shape*. Square 4159 (0.39 km from Bocconi University) has office-hours ratio 1.45, weekday peak 12:00, and weekend activity 0.59× weekday. Square 4556 (Navigli nightlife) peaks at 22:00 with weekend traffic 1.14× weekday — the only area where nights exceed office hours. Square 5161 (Duomo, rank 1) peaks afternoons with weekend 1.38× weekday; square 5259, 0.2 km away, is office-dominated (weekend 0.43×). Per-area scaling and unchanged cross-area transfer are therefore substantive tests, not formalities.

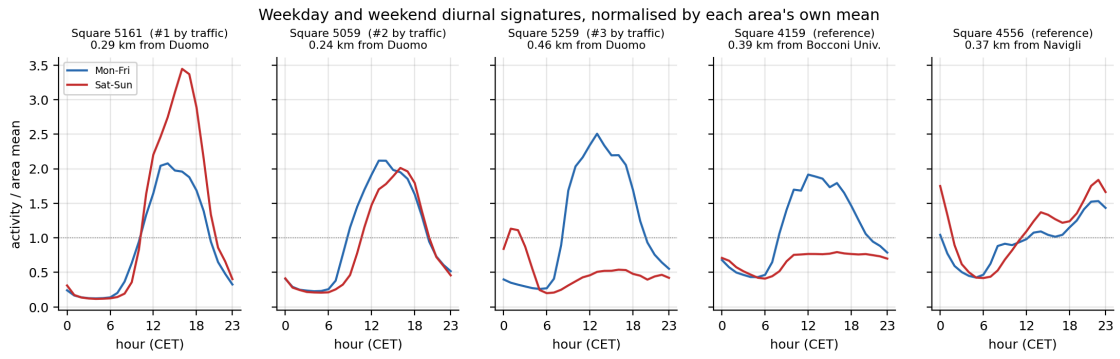


Figure 3. Weekday/weekend diurnal signatures, normalised by area mean.

4.3 Autocorrelation and periodicity

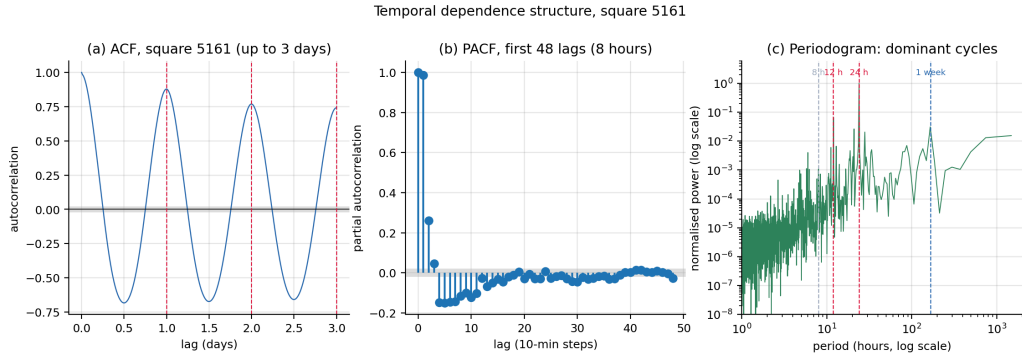


Figure 4. Square 5161. (a) ACF to three days. (b) PACF (48 lags). (c) Periodogram.

lag_label	lag_steps	acf
10 min	1	0.987
1 hour	6	0.939
6 hours	36	0.009891
12 hours	72	-0.683
1 day	144	0.878
2 days	288	0.77
3 days	432	0.741

Lag-1 ACF = 0.987; lag 144 = 0.878; lag 72 = -0.683 (day/night antiphase). Periodogram peak at 24 h with 12 h harmonic; weaker weekly peak ~6.9 days. PACF: 0.99 at lag 1, 0.26 at lag 2, 0.04 at lag 3 — low AR order after seasonal differencing. Persistence is a serious baseline; lookback 144 motivates neural input windows and TCN receptive-field tuning.

4.4 Stationarity and decomposition

series	ADF_stat	ADF_p	ADF_verdict	KPSS_stat	KPSS_p	KPSS_verdict	std
raw	-19.032	0	stationary	0.237	0.1	stationary	1382
log1p	-22.743	0	stationary	0.571	0.026	non-stationary	1.203
first difference	-15.16	6.603e-28	stationary	0.003337	0.1	stationary	221.16
seasonal difference (lag 144)	-11.833	7.909e-22	stationary	0.069	0.1	stationary	639.05
log1p + seasonal difference	-9.956	2.444e-17	stationary	0.153	0.1	stationary	0.417

Raw-series ADF/KPSS both indicate stationarity, but neither detects deterministic seasonality — the ACF/periodogram do. After log1p + seasonal difference at lag 144, both tests agree on stationarity (KPSS 0.153).

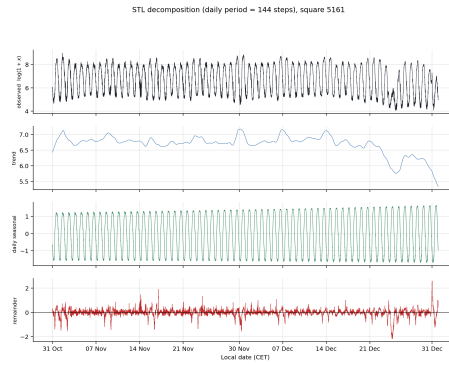


Figure 5. STL [17] decomposition of `log1p` activity, square 5161, period 144. Trend falls after 21 December (Christmas).

quantity	value
Trend strength F_T (0-1)	0.519
Daily seasonal strength F_S (0-1)	0.927
$\text{Var}(\text{observed})$, log scale	1.447
$\text{Var}(\text{trend})$	0.092
$\text{Var}(\text{seasonal})$	1.289
$\text{Var}(\text{remainder})$	0.098
Remainder std as % of observed std	26.07

Daily seasonal strength = **0.927**; trend strength = 0.519; remainder SD = 26.1% of observed SD on log scale — an approximate error floor. Calendar anomalies include Christmas Day (z to -6.85), New Year (z to $+8.31$), and All Saints' weekend. Test week 16–22 December precedes the severest holiday anomalies but contains ordinary weekly cycles that lag-144 seasonal terms cannot represent.

Consequence. Seasonal differencing at lag 144 on `log1p` supports $\text{SARIMA}(p,d,q)(0,1,0)[144]$; PACF bounds low p , q . Calendar-driven departures and missing day-of-week inputs limit univariate models — the failure analysis in Section 6.4 confirms this.

5. Methodology

5.1 Task and evaluation protocol

One-step-ahead forecasting: given observations through t , predict $t+1$ (10 minutes). Evaluated over all 1,008 intervals of **16–22 December 2013** on areas 5161, 5059, 5259. Data split chronologically, never shuffled [11]:

Partition	Dates	Intervals	Role
Train	1 Nov – 8 Dec 2013	5,472	fitting; scaler stats
Validation	9 – 15 Dec 2013	1,008	hyperparameter selection
Test	16 – 22 Dec 2013	1,008	reported results

Validation and test predictions condition on observations immediately before each forecast, including from earlier partitions — this mirrors operational use and is not leakage. No target from validation or test influences fitted parameters, hyperparameter choices, or scaler statistics. SARIMA coefficients are estimated on training data and held fixed; neural networks select weights by validation loss and never see the test week during training. Tuning on square 5161 only; selected configs applied unchanged to all areas so cross-area differences measure generalisation.

5.2 Preprocessing and metrics

Transform: $y = \log_{1p}(x)$, standardised per area on training only, inverted with $\hat{x} = \max(\exp(\sigma \cdot \hat{y} + \mu), 0)$. `log1p` handles heavy tails, heteroskedasticity [11], and exact zeros (0.019% of cells).

Windowing: Neural models use sliding windows of length L over scaled series; candidates 36, 144, 288 steps motivated by ACF. SARIMA receives seasonally differenced `log1p` directly.

Metrics computed in original activity units after inversion:

Metric	Role
MAE, RMSE, MAPE	Required
sMAPE	Reported; unreliable near zero
MASE	Primary scaled metric; denominator = seasonal-naive MAE on the evaluation partition , not Hyndman's in-sample one-step naive [16] — comparable across areas here but not to published MASE
R^2 , $\text{RMSE} \div \text{MAE}$	Reference; ratio > 1 indicates peak-dominated error

Baselines: Persistence $\hat{x}(t+1)=x(t)$; seasonal naive $\hat{x}(t+1)=x(t+1-144)$ (MASE denominator).

5.3 Models

Model	Family	History aggregation
SARIMA(p,d,q)(0,1,0)[144]	Linear	Seasonal differencing + ARMA
LSTM	Recurrent [12]	Gated hidden state
TCN	Convolutional [13], [14]	Dilated causal convolutions

SARIMA is the honest linear test; LSTM is the recurrent standard in cellular traffic [7], [9]; TCN is the structural contrast with fixed receptive field.

5.4 Specifications

SARIMA. `SARIMA(p,d,q)(0,1,0)[144]` on `log1p`. Seasonal difference applied explicitly ($z_t = y_t - y_{t-144}$); fit `ARIMA(p,d,q)` to z ; invert via $\hat{x}_{t+1} = \expm1(\hat{z}_{t+1} + y_{t+1-144})$ because `statsmodels` [31] state-space with non-zero seasonal P or Q carries ~ 144 state variables and makes order search impractical on two CPU cores. This is mathematically equivalent to `SARIMA(p,d,q)(0,1,0)[144]` with $P = Q = 0$ — a real restriction, accepted because seasonal differencing alone achieves stationarity (Section 4.4). ML on training partition; `trend='n'` (not the library default, which would silently add a constant); one-step Kalman predictions with fixed coefficients (`dynamic=False`).

LSTM. One- or two-layer LSTM [12], linear head on final hidden state, input `(batch, L, 1)`. PyTorch [32]; Adam [27]; Huber loss [26]; gradient clip 1.0; `ReduceLROnPlateau`; early stopping on validation MAE.

TCN. Residual blocks with dilation 2^i , causal convolutions [13], [14]. Receptive field: $RF = 1 + 2(k-1)(2^L - 1)$; with $k = 3$, four levels $\rightarrow RF = 61 (< 144)$; six levels $\rightarrow RF = 253$. Same training protocol as LSTM.

5.5 Tuning design

Sequential search (not grid), one change at a time, logged to `results/tuning_<model>_sq5161.csv`. Tuning: 30 epochs max, patience 5, 420 s wall-clock cap; final retrain: 60 epochs, patience 8, 900 s. `RANDOM_SEED = 42`; PyTorch pinned to 2 threads. Wall-clock caps mean selection is best *under budget*, not necessarily asymptotically optimal.

6. Results and Discussion

6.1 Hyperparameter experiments

All tuning on square 5161, validation week 9–15 December.

SARIMA

experiment	rationale	model	square_id	partition	MAE	RMSE	MAPE_%	sMAPE_%	R2	RMSE/MAE	MASE	train_seconds	inference_seconds
1	Baseline: a single AR term on the seasonally differenced series.	SARIMA	5161	val	154.711	246.865	11.428	11.267	0.974	1.596	0.525	0.863	0.067
2	A pure MA term at the same complexity, to see which the data prefers.	SARIMA	5161	val	222.226	378.251	14.606	14.286	0.938	1.702	0.754	0.662	0.101
3	ARMA(1,1): the standard first upgrade when both ACF and PACF decay.	SARIMA	5161	val	136.333	210.59	10.19	10.066	0.981	1.545	0.462	0.938	0.096
4	The PACF shows significant mass at lag 2; add a second AR term.	SARIMA	5161	val	137.011	211.411	10.247	10.123	0.981	1.543	0.465	1.765	0.12
5	Probe whether AR order keeps helping or starts to overfit.	SARIMA	5161	val	137.023	211.242	10.254	10.129	0.981	1.542	0.465	3.91	0.24
6	Trade an AR term for an extra MA term at equal parameter count.	SARIMA	5161	val	136.933	210.792	10.248	10.123	0.981	1.539	0.464	9.383	0.098
7	Add a non-seasonal difference to test for residual drift.	SARIMA	5161	val	136.632	207.997	10.411	10.287	0.981	1.522	0.463	2.168	0.113
8	Re-test the best order with a constant, to check whether the seasonally differenced series carries any residual mean offset. Worth testing explicitly because statsmodels includes a constant by default and it is easy to fit one unintentionally.	SARIMA	5161	val	137.045	211.415	10.251	10.123	0.981	1.543	0.465	4.906	0.464

PACF supports low orders. Pure MA(1) fails vs AR(1); ARMA(1,1) gives largest gain (MAE 136.3, MASE 0.462). Beyond ARMA(1,1), validation MAE flat (136.3–137.0) while AIC still improves — held-out MAE selects **ARMA(1,1) on seasonally differenced log1p** (3 parameters). Non-seasonal difference and constant term add nothing.

LSTM

experiment	rationale	model	square_id	partition	MAE	RMSE	MAPE_%	sMAPE_%	R2	RMSE/MAE	MASE	train_seconds	inference_seconds	ir
1	Start small: 6 hours of history, 32 units. Establishes whether short-range continuity alone already beats the baseline.	LSTM	5161	val	108.213	166.468	7.852	7.971	0.988	1.538	0.367	101.62	0.052	
2	Extend the window to a full day (144 steps). The ACF shows a strong peak at lag 144, so the model should be able to use yesterday's same-time value.	LSTM	5161	val	112.522	173.409	8.343	8.257	0.987	1.541	0.382	239.949	0.332	
3	Double the hidden width at the better lookback to test whether capacity, not context, is the binding constraint.	LSTM	5161	val	107.274	164.035	8.013	8.004	0.988	1.529	0.364	231.908	0.275	
4	Add depth with light dropout: a second layer can compose short-range and daily features hierarchically.	LSTM	5161	val	130.064	195.437	9.755	9.553	0.984	1.503	0.441	667.336	5.207	
5	Larger batches with a proportionally higher learning rate. Tests whether the earlier runs were optimisation-limited rather than capacity-limited, at roughly half the epoch cost.	LSTM	5161	val	113.964	171.545	9.248	8.992	0.987	1.505	0.387	455.831	1.961	
6	Two days of history, to check whether the weekly structure visible in the EDA is reachable through a longer window.	LSTM	5161	val	144.693	217.255	10.553	10.477	0.98	1.502	0.491	453.058	1.799	

Extending lookback 36 → 144 worsened MAE (108.2 → 112.5); doubling hidden width at 144 recovered and improved (107.3, MASE 0.364) — longer windows need more capacity. Two-layer and 288-step runs hit the 420 s cap (truncated, not converged). Best: **64 hidden units, 144-step window**.

TCN initial search

experiment	rationale	model	square_id	partition	MAE	RMSE	MAPE_%	sMAPE_%	R2	RMSE/MAE	MASE	train_seconds	inference_seconds
1	Receptive field 61 steps (~10 h): deliberately shorter than one day, to demonstrate that the daily lag is out of reach.	TCN	5161	val	103.172	158.36	7.701	7.681	0.989	1.535	0.35	408.731	0.393
2	Two more dilation levels push the receptive field to 253 steps, which covers the lag-144 daily peak identified by the ACF.	TCN	5161	val	106.134	159.585	7.846	7.831	0.989	1.504	0.36	450.03	1.318
3	With the receptive field sufficient, widen the channels to test whether representational capacity is now the limit.	TCN	5161	val	110.483	171.53	8.286	8.189	0.987	1.552	0.375	426.016	10.976
4	Remove dropout: with only ~5k training windows the noise it injects may cost more than the regularisation it buys.	TCN	5161	val	106.098	163.7	7.604	7.758	0.988	1.543	0.36	445.883	2.568
5	Receptive field 187 with one fewer dilation level, trading depth for kernel width. The deepest level dominates cost because it convolves the most heavily padded sequence, so this is markedly cheaper at a comparable receptive field.	TCN	5161	val	111.977	173.559	8.805	8.596	0.987	1.55	0.38	454.782	7.81
6	Receptive field 509 over a two-day window, matching the longest-context LSTM configuration for a fair comparison.	TCN	5161	val	111.229	169.046	8.071	8.054	0.988	1.52	0.377	474.637	10.624

Only the shallowest run (RF = 61) completed by early stopping; deeper configs hit the wall-clock cap at 5–15 epochs, confounding receptive field with training effort. A controlled follow-up was required.

TCN receptive-field study

Fixed: channels, batch size, LR, 144-step lookback, **24 epochs each**, no early stopping. Depth varies RF: 61, 125, 253, 509 steps. Single seed (42); four-level config reproduced 103.17 exactly — pipeline deterministic, seed sensitivity unmeasured.

experiment	rationale	model	square_id	partition	MAE	RMSE	MAPE_%	sMAPE_%	R2	RMSE/MAE	MASE	train_seconds	inference_seconds	info
1	Receptive field 61 steps (~10 h): cannot reach the lag-144 daily peak.	TCN	5161	val	103.172	158.36	7.701	7.681	0.989	1.535	0.35	1352	3.16	
2	Receptive field 125 steps (~21 h): just short of one full day.	TCN	5161	val	109.605	167.054	7.899	8.065	0.988	1.524	0.372	2110	0.972	
3	Receptive field 253 steps (~42 h): comfortably covers the daily lag.	TCN	5161	val	104.546	159.839	7.723	7.747	0.989	1.529	0.354	987.646	1.062	
4	Receptive field 509 steps (~85 h): covers two days, well beyond the window.	TCN	5161	val	102.473	154.714	7.694	7.647	0.99	1.51	0.347	1169	1.21	

Validation MAE: 61 $\rightarrow$ 103.17, 125 $\rightarrow$ 109.61, 253 $\rightarrow$ 104.55, 509 $\rightarrow$ 102.47. Ordering is **non-monotonic** (125 worst, worse than both neighbours by 5–6 MAE units); since depth is the only variable and epochs are matched, the middle configuration's poor performance bounds run-to-run variation at ~5 MAE units. Best–worst gap $0.70 \ll 5$ — **no measurable RF effect** at one-step horizon. A 61-step field that provably cannot see "same time yesterday" matches one covering three and a half days. Lag-144 seasonality is real but largely redundant when lag-1 ACF = 0.987: recent observations already imply the daily cycle at a ten-minute horizon, though it remains decisive for SARIMA, which must encode it explicitly. Selected **levels = 4** (RF = 61, 5,601 parameters) for parsimony over levels = 7 (10,305 parameters, +84% params for 0.7 MAE).

6.2 Final test-week results

Configurations: SARIMA(1,0,1)(0,1,0)[144]; LSTM 64 hidden, 144 window; TCN 16 channels, 4 levels — unchanged across areas.

Square 5161:

model	MAE	MAPE_%	RMSE	sMAPE_%	MASE	R2	RMSE/MAE
Persistence	92.802	9.194	134.878	9.096	0.274	0.99	1.453
Seasonal naive	338.594	25.94	619.04	22.828	1	0.793	1.828
SARIMA	108.035	10.574	163.051	10.328	0.319	0.986	1.509
LSTM	93.118	8.364	147.011	8.175	0.275	0.988	1.579
TCN	78.492	7.496	119.578	7.367	0.232	0.992	1.523

Square 5059:

model	MAE	MAPE_%	RMSE	sMAPE_%	MASE	R2	RMSE/MAE
Persistence	81.517	7.959	114.375	7.909	0.475	0.985	1.403
Seasonal naive	171.736	18.018	245.87	16.621	1	0.931	1.432
SARIMA	89.292	8.626	127.904	8.509	0.52	0.981	1.432
LSTM	67.231	6.451	97.879	6.42	0.392	0.989	1.456
TCN	68.867	6.874	99.478	6.73	0.401	0.989	1.444

Square 5259:

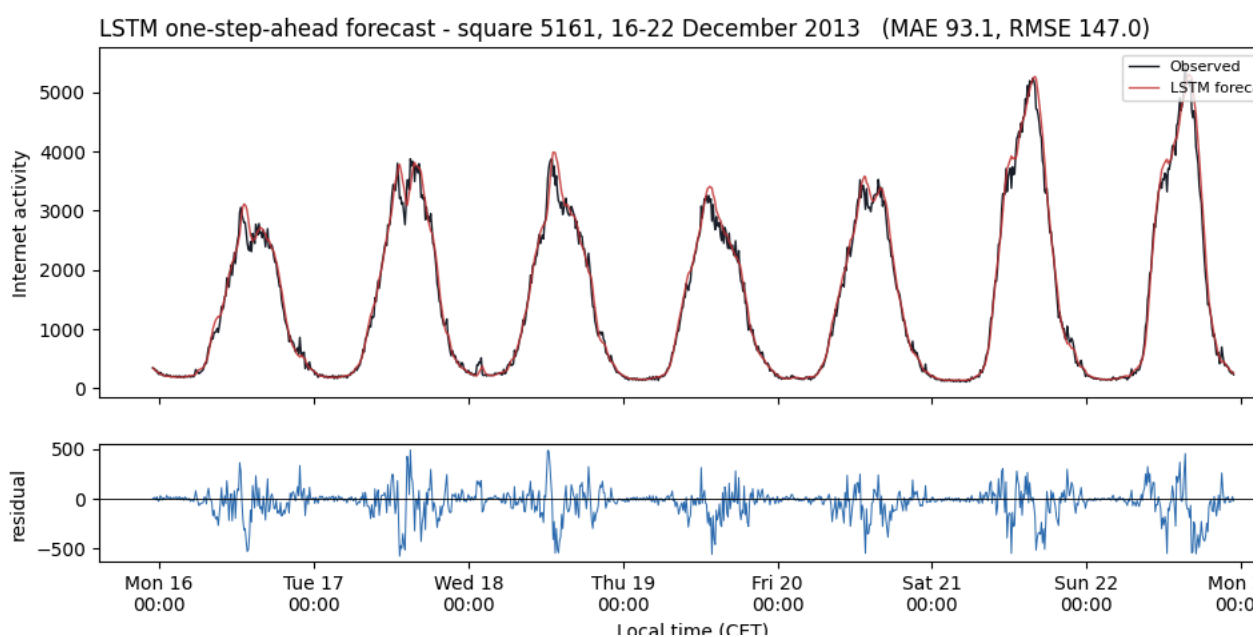
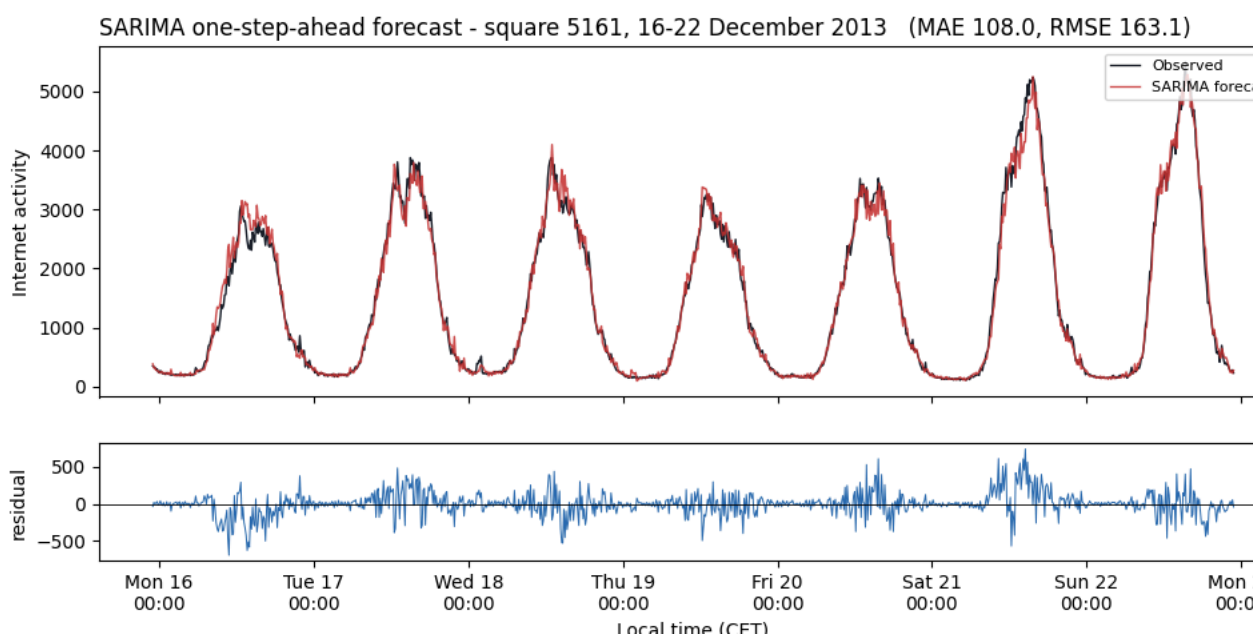
model	MAE	MAPE_%	RMSE	sMAPE_%	MASE	R2	RMSE/MAE
Persistence	75.968	8.115	109.578	8.074	0.162	0.99	1.442
Seasonal naive	470.32	71.621	861.621	42.613	1	0.409	1.832
SARIMA	81.776	8.722	123.334	8.66	0.174	0.988	1.508
LSTM	67.321	7.214	96.814	7.151	0.143	0.992	1.438
TCN	64.745	7.112	93.3	6.934	0.138	0.993	1.441

Seasonal naive is catastrophically weak (MAE 338.6, 171.7, 470.3 vs persistence 92.8, 81.5, 76.0), so $MASE \ll 1$ for all models. **Persistence is the binding baseline** (lag-1 ACF 0.987):

model	sq5059	sq5161	sq5259	mean	spread
Persistence	1	1	1	1	0
Seasonal naive	2.107	3.649	6.191	3.982	4.084
SARIMA	1.095	1.164	1.077	1.112	0.088
LSTM	0.825	1.003	0.886	0.905	0.179
TCN	0.845	0.846	0.852	0.848	0.0074

SARIMA is 7.6–16.4% worse than persistence on all areas. This is not a tuning failure: the order search tested eight configurations bounded by the PACF and flattened before the search boundary. Seasonal differencing at lag 144 makes the series stationary but discards the short-range information that dominates at a ten-minute horizon. **TCN** improves 14.8–15.5% with spread **0.007** across areas differing fivefold in level and inverting weekend behaviour (5161 weekend 1.38× weekday; 5259 0.43×). **LSTM** mean ratio 0.905 but spread 0.179 (25× TCN's): best on 5059 (67.2 vs TCN 68.9), ties persistence on 5161 (93.1 vs 92.8). LSTM on 5161 was the only final run hitting its wall-clock budget (epoch 22/40); its best epoch matched tuning, but convergence cannot be asserted for that area alone.

Forecast plots



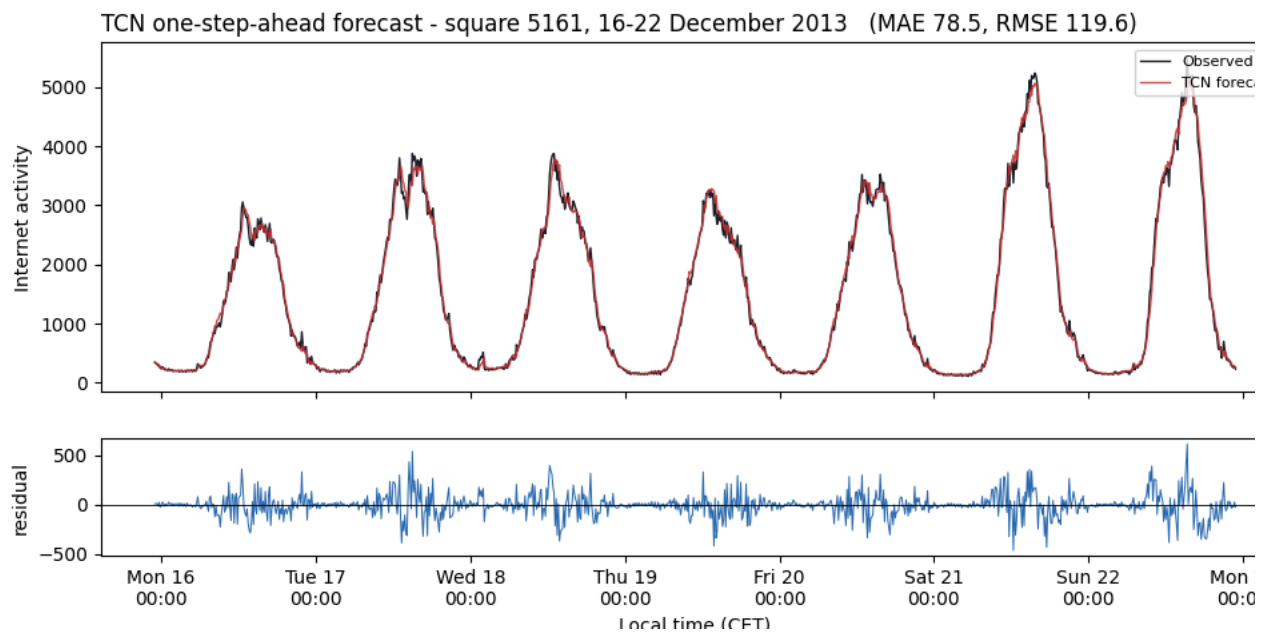
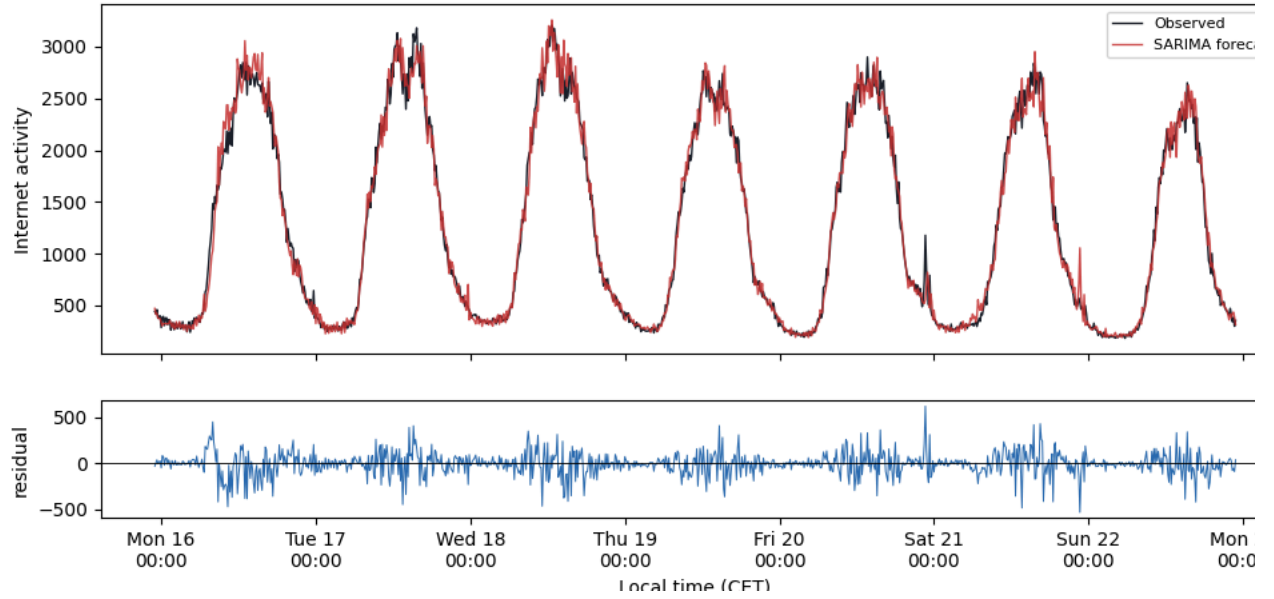
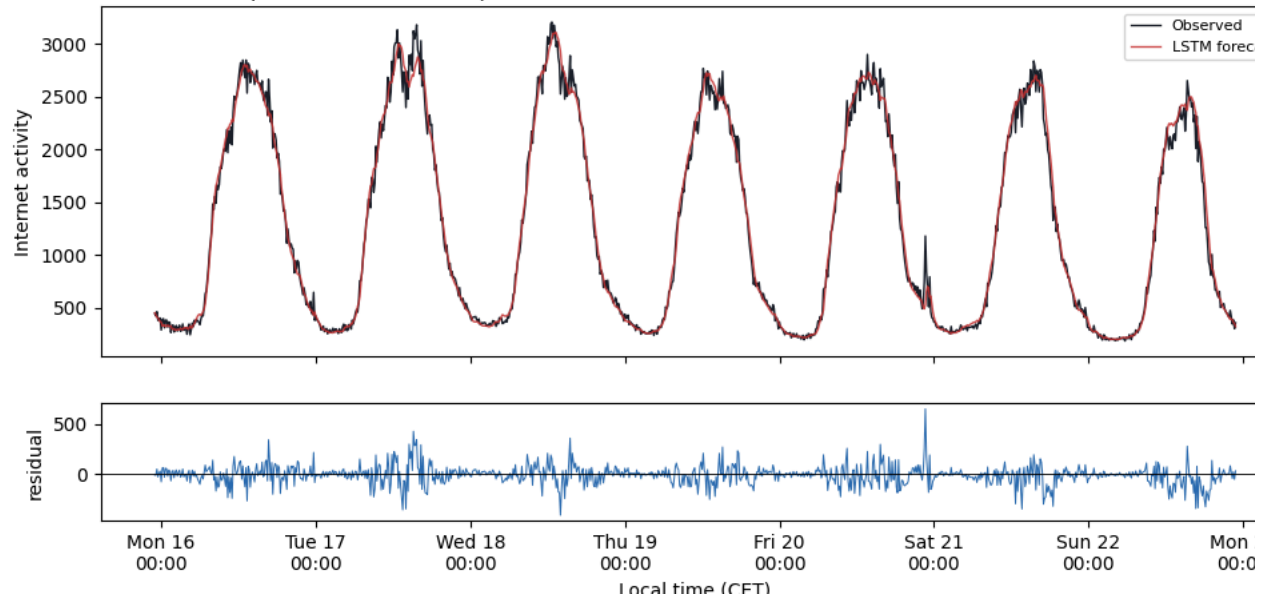


Figure 6. Square 5161, 16–22 December: SARIMA, LSTM, TCN forecasts with residuals.

SARIMA one-step-ahead forecast - square 5059, 16-22 December 2013 (MAE 89.3, RMSE 127.9)



LSTM one-step-ahead forecast - square 5059, 16-22 December 2013 (MAE 67.2, RMSE 97.9)



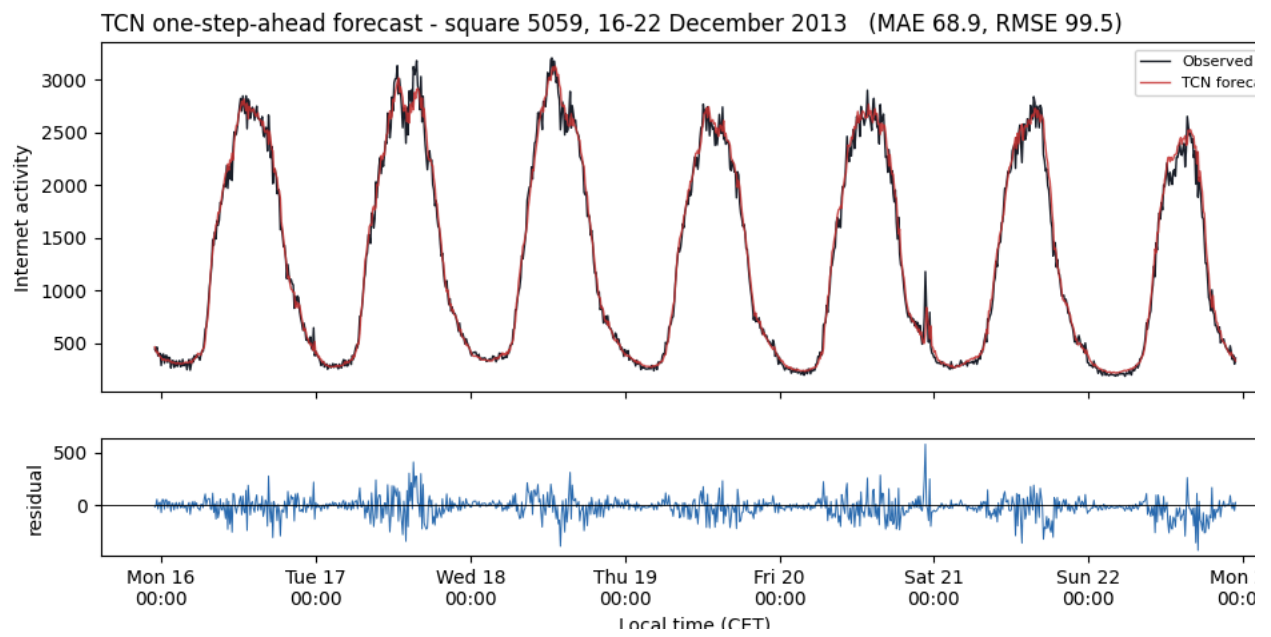
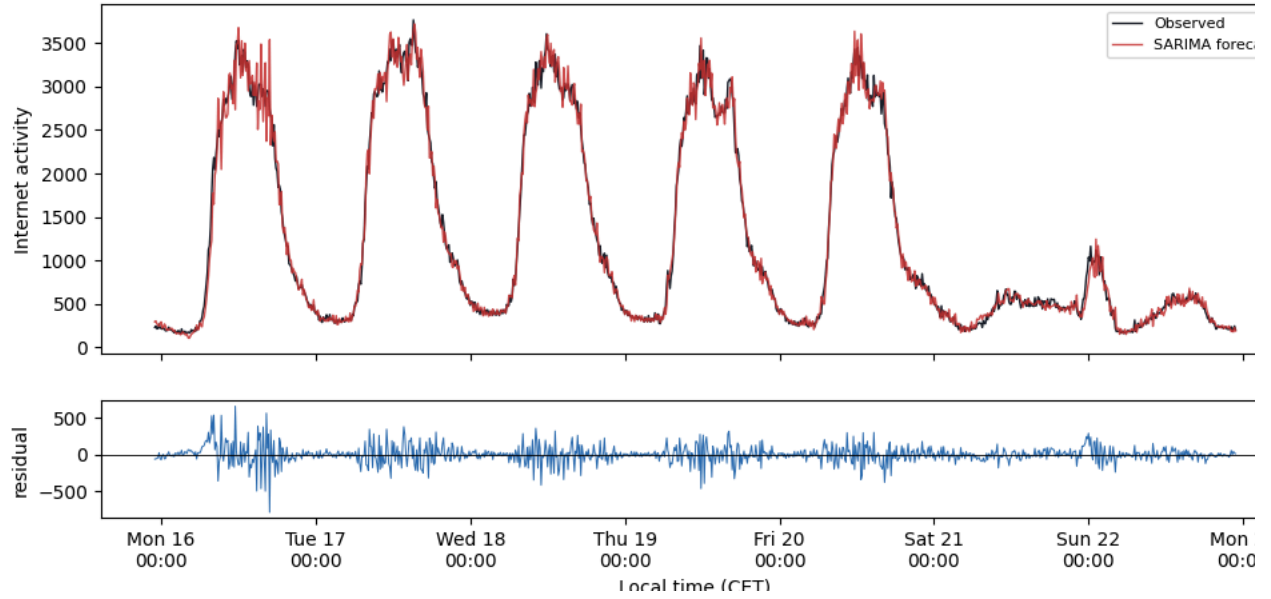
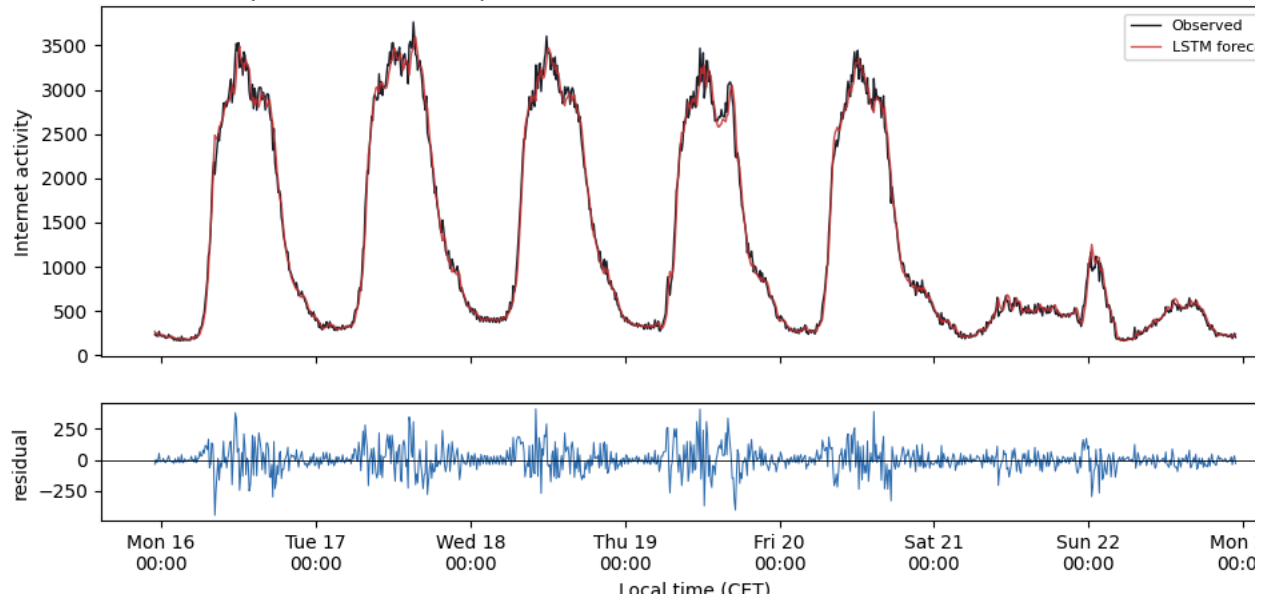


Figure 7. Square 5059, same week.

SARIMA one-step-ahead forecast - square 5259, 16-22 December 2013 (MAE 81.8, RMSE 123.3)



LSTM one-step-ahead forecast - square 5259, 16-22 December 2013 (MAE 67.3, RMSE 96.8)



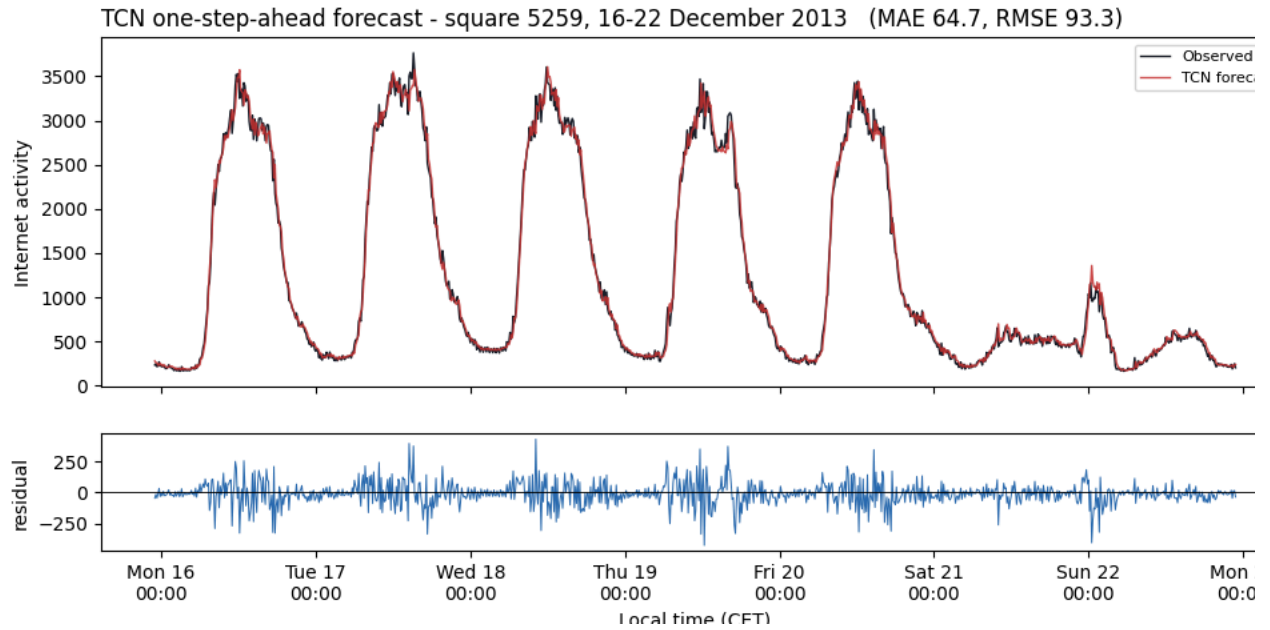


Figure 8. Square 5259, same week.

All models track the diurnal cycle (R^2 0.981–0.993); residuals expand at daily peaks and weekends. $RMSE \div MAE = 1.44\text{--}1.58$ (models) vs 1.83 (seasonal naive on 5161, 5259) — error is peak-concentrated.

6.3 Computational cost

model	train_seconds_mean	train_seconds_min	train_seconds_max	inference_seconds_week	inference_ms_per_step	n_parameters	best_epoch
Persistence	0	0	0	0.065	0.065	0	-
Seasonal naive	0	0	0	0.066	0.065	0	-
SARIMA	1.162	0.743	1.666	0.115	0.114	3	-
LSTM	1005	266.703	1929	0.687	0.682	17217	27.000
TCN	644.485	193.046	1371	0.371	0.368	5601	26.333

Hardware: i5-6300U, 7.9 GB RAM, no GPU, 2 PyTorch threads (results/hardware.json).

Timing caveat: runs occurred on a loaded laptop (~466 MB free during evaluation). Same LSTM config: ~10.5 s/epoch (tuning) vs ~88 s/epoch (final) — eightfold difference with unchanged architecture. Order-of-magnitude comparisons only.

Nonetheless: **SARIMA** fits in 0.74–1.67 s (3 parameters) but loses to persistence. **TCN** mean training 644 s vs LSTM 1,005 s, with 5,601 vs 17,217 parameters — better accuracy and lower cost. **Inference** ≤ 0.68 ms/step — not operationally binding. Wall-clock budgets make strict reproducibility impossible; predictions were archived to results/predictions_square_*.csv for regeneration without retraining (post-hoc; reported numbers are from the original run).

6.4 Failure analysis

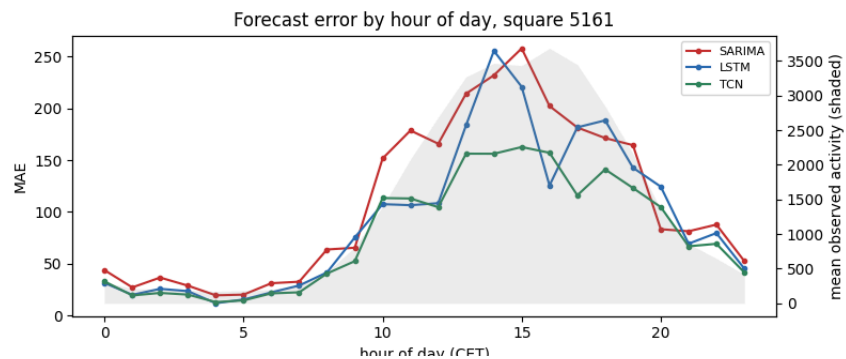


Figure 9. Square 5161: MAE by hour with mean activity shaded. Models differ only at peak (13:00–16:00).

Overnight (00:00–06:00) all models ~15–45 MAE; at peak SARIMA ~258, LSTM ~256, TCN ~163. TCN's advantage is entirely at the daily peak.

Worst six-hour windows (all areas: midday–evening peak):

Area	Worst window	SARIMA	LSTM	TCN
5161	Sat 21 Dec, 13:10–19:00	304.0 (2.81×)	201.0 (2.16×)	148.2 (1.89×)
5059	Tue 17 Dec, 12:20–18:10	158.6 (1.78×)	173.7 (2.58×)	164.8 (2.39×)
5259	Mon 16 Dec, 12:20–18:10	221.7 (2.71×)	131.0 (1.95×)	118.6 (1.83×)

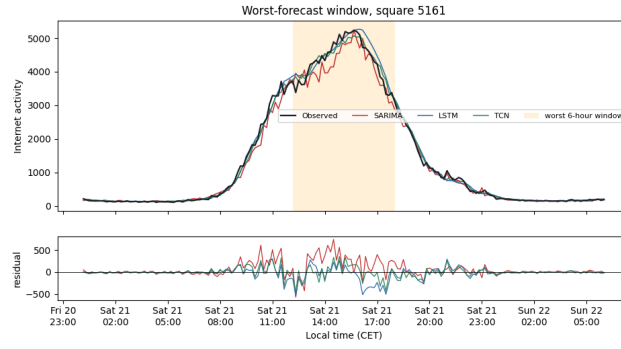


Figure 10. Square 5161 worst window: Saturday 21 December, 13:10–19:00 — highest-activity period after 48% Fri→Sat jump.

Daily peaks Mon–Fri: 3,057–3,877; Sat 5,238; Sun 5,496. This is **not** beyond training range (training Saturday peak 8,044; Sat 21 Dec is lower than several training Saturdays). STL trend *falls* after 21 December. The failure is the **day-of-week transition**: Friday peak 3,530 → Saturday 5,238 (+48%). No model receives day-of-week input.

- **SARIMA**: bias −207 — anchored to yesterday (Friday), systematically under-predicts.
- **LSTM**: bias +112 — tracks ascent but overshoots descent.
- **TCN**: bias +15 — lowest window error, largely unbiased.

6.5 Summary

	SARIMA	LSTM	TCN
Mean ratio to persistence	1.112	0.905	0.848
Cross-area spread	0.088	0.179	0.007
Beats persistence	0/3	2/3	3/3
Parameters	3	17,217	5,601
Mean train time	1.2 s	1,005 s	644 s

The TCN is preferred on accuracy, cross-area stability, and efficiency. Results align with literature once horizon is considered: Zhang and Patras [2] report larger gains at long horizons; Makridakis *et al.* [21] and Wang *et al.* [6] find simple methods competitive; Zeng *et al.* [29] show persistence strength depends on horizon. Zhang *et al.* [28] use a 3.19M-parameter spatio-temporal hybrid against neural baselines only — complementary to this univariate persistence comparison.

7. Conclusion and Future Work

7.1 Findings

We compared SARIMA, LSTM, and TCN for one-step-ahead Internet traffic forecasting in three Milan grid areas (16–22 December 2013).

1. **Baseline choice matters.** MASE $\ll 1$ vs seasonal naive masks that persistence is the binding benchmark; SARIMA is 7.6–16.4% worse than persistence everywhere.
2. **TCN is most reliable.** 14.8–15.5% gain over persistence, spread 0.007, despite unchanged transfer across areas with inverted weekend behaviour. LSTM mean gain similar but spread 25× larger.
3. **Receptive field irrelevant at one step.** Matched-epoch study: no effect from RF 61–509; daily seasonality is decisive for SARIMA, redundant for networks conditioned on recent history.
4. **Failures are calendar/design-driven.** Errors concentrate at peaks; worst window is Fri→Sat transition (+48%), not an unprecedented pre-Christmas peak. Missing day-of-week input explains the largest miss.

7.2 Limitations

Single seed per config; wall-clock budgets (one truncated LSTM final run); contaminated timing measurements; three adjacent high-traffic areas only; SARIMA restricted to $P = Q = 0$; univariate inputs; symmetric metrics [8] that under-weight SARIMA's peak under-prediction.

7.3 Future work

1. **Day-of-week / holiday features** — targets the observed +48% transition failure directly.
2. **Multi-seed repeats** on an unloaded machine.
3. **Horizon sweep** (1, 6, 36, 144 steps) to test when RF and seasonality matter [2].
4. **Areas across the traffic distribution** (median, bottom decile).
5. **Spatio-temporal models** exploiting spatial decay [4], [15] — requires re-ingesting discarded channels/dimensions.
6. **Asymmetric cost training** following DeepCog [8].

8. Academic Integrity

This is an individual submission. Library documentation, textbooks, and published papers listed in Section 9 were used as learning resources while developing the methods. Occasional use of a programming assistant was limited to clarifying language or library usage where needed; it was not used as a substitute for understanding the problem, designing the experiments, interpreting the results, or writing the scientific argument of this report.

All numerical results in this report were produced by the code and experiments in the accompanying repository [33]. I take responsibility for the work submitted under my name and can explain and justify the data handling, modelling choices, methodology, results, and conclusions.

9. References

- [1] G. Barlacchi, M. De Nadai, R. Larcher, A. Casella, C. Chitic, G. Torrisi, F. Antonelli, A. Vespignani, A. Pentland, and B. Lepri, "A multi-source dataset of urban life in the city of Milan and the Province of Trentino," *Scientific Data*, vol. 2, art. no. 150055, Oct. 2015, doi: 10.1038/sdata.2015.55.
- [2] C. Zhang and P. Patras, "Long-term mobile traffic forecasting using deep spatio-temporal neural networks," in *Proc. 19th ACM Int. Symp. Mobile Ad Hoc Networking and Computing (MobiHoc '18)*, Los Angeles, CA, USA, Jun. 2018, pp. 231–240, doi: 10.1145/3209582.3209606.
- [3] C. Zhang, H. Zhang, D. Yuan, and M. Zhang, "Citywide cellular traffic prediction based on densely connected convolutional neural networks," *IEEE Communications Letters*, vol. 22, no. 8, pp. 1656–1659, Aug. 2018, doi: 10.1109/LCOMM.2018.2841832.
- [4] C. Zhang, H. Zhang, J. Qiao, D. Yuan, and M. Zhang, "Deep transfer learning for intelligent cellular traffic prediction based on cross-domain big data," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 6, pp. 1389–1401, Jun. 2019, doi: 10.1109/JSAC.2019.2904363.
- [5] C.-W. Huang, C.-T. Chiang, and Q. Li, "A study of deep learning networks on mobile traffic forecasting," in *Proc. IEEE 28th Annu. Int. Symp. Personal, Indoor, and Mobile Radio Communications (PIMRC)*, Montreal, QC, Canada, Oct. 2017, pp. 1–6, doi: 10.1109/PIMRC.2017.8292737.
- [6] X. Wang, Z. Zhou, F. Xiao, K. Xing, Z. Yang, Y. Liu, and C. Peng, "Spatio-temporal analysis and prediction of cellular traffic in metropolis," *IEEE Transactions on Mobile Computing*, vol. 18, no. 9, pp. 2190–2202, Sep. 2019, doi: 10.1109/TMC.2018.2870135.
- [7] H. D. Trinh, L. Giupponi, and P. Dini, "Mobile traffic prediction from raw data using LSTM networks," in *Proc. IEEE 29th Annu. Int. Symp. Personal, Indoor and Mobile Radio Communications (PIMRC)*, Bologna, Italy, Sep. 2018, pp. 1827–1832, doi: 10.1109/PIMRC.2018.8581000.
- [8] D. Bega, M. Gramaglia, M. Fiore, A. Banchs, and X. Costa-Pérez, "DeepCog: Cognitive network management in sliced 5G networks with deep learning," in *Proc. IEEE INFOCOM 2019 – IEEE Conf. Computer Communications*, Paris, France, Apr. 2019, pp. 280–288, doi: 10.1109/INFOCOM.2019.8737488.
- [9] C. Zhang, P. Patras, and H. Haddadi, "Deep learning in mobile and wireless networking: A survey," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2224–2287, 3rd Quart. 2019, doi: 10.1109/COMST.2019.2904897.
- [10] Y. Shu, M. Yu, J. Liu, and O. W. W. Yang, "Wireless traffic modeling and prediction using seasonal ARIMA models," in *Proc. IEEE Int. Conf. Communications (ICC)*, May 2003, vol. 3, pp. 1675–1679, doi: 10.1109/ICC.2003.1203886.
- [11] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: <https://otexts.com/fpp3/>
- [12] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997, doi: 10.1162/neco.1997.9.8.1735.
- [13] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," arXiv:1803.01271 [cs.LG], Mar. 2018, doi: 10.48550/arXiv.1803.01271. (Technical report; not a peer-reviewed conference paper.)
- [14] A. van den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, and K. Kavukcuoglu, "WaveNet: A generative model for raw audio," arXiv:1609.03499 [cs.SD], Sep. 2016, doi: 10.48550/arXiv.1609.03499.
- [15] X. Shi, Z. Chen, H. Wang, D.-Y. Yeung, W.-K. Wong, and W.-C. Woo, "Convolutional LSTM network: A machine learning approach for precipitation nowcasting," in *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, Montreal, QC, Canada, Dec. 2015. [Online]. Available: <https://arxiv.org/abs/1506.04214>
- [16] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," *International Journal of Forecasting*, vol. 22, no. 4, pp. 679–688, Oct.–Dec. 2006, doi: 10.1016/j.ijforecast.2006.03.001.
- [17] R. B. Cleveland, W. S. Cleveland, J. E. McRae, and I. Terpenning, "STL: A seasonal-trend decomposition procedure based on loess," *Journal of Official Statistics*, vol. 6, no. 1, pp. 3–73, 1990.
- [18] X. Wang, K. Smith, and R. Hyndman, "Characteristic-based clustering for time series data," *Data Mining and Knowledge Discovery*, vol. 13, no. 3, pp. 335–364, Nov. 2006, doi: 10.1007/s10618-005-0039-x.
- [19] D. A. Dickey and W. A. Fuller, "Distribution of the estimators for autoregressive time series with a unit root," *Journal of the American Statistical Association*, vol. 74, no. 366, pp. 427–431, Jun. 1979, doi: 10.1080/01621459.1979.10482531.
- [20] D. Kwiatkowski, P. C. B. Phillips, P. Schmidt, and Y. Shin, "Testing the null hypothesis of stationarity against the alternative of a unit root: How sure are we that economic time series have a unit root?," *Journal of Econometrics*, vol. 54, no. 1–3, pp. 159–178, Oct.–Dec. 1992, doi: 10.1016/0304-4076(92)90104-Y.
- [21] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "Statistical and machine learning forecasting methods: Concerns and ways forward," *PLoS ONE*, vol. 13, no. 3, art. no. e0194889, Mar. 2018, doi: 10.1371/journal.pone.0194889.
- [22] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M4 Competition: 100,000 time series and 61 forecasting methods," *International Journal of Forecasting*, vol. 36, no. 1, pp. 54–74, Jan.–Mar. 2020, doi: 10.1016/j.ijforecast.2019.04.014.
- [23] GSMA Intelligence, "Going green: Benchmarking the energy efficiency of mobile networks (second edition)," GSMA Intelligence, London, U.K., Feb. 2023. (Industry report; not peer-reviewed.)
- [24] L. M. P. Larsen, H. L. Christiansen, S. Ruepp, and M. S. Berger, "Toward greener 5G and beyond radio access networks—A survey," *IEEE Open Journal of the Communications Society*, vol. 4, pp. 768–797, 2023, doi: 10.1109/OJCOMS.2023.3257889.
- [25] J. Wu, Y. Zhang, M. Zukerman, and E. K.-N. Yung, "Energy-efficient base-stations sleep-mode techniques in green cellular networks: A survey," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 2, pp. 803–826, 2nd Quart. 2015, doi: 10.1109/COMST.2015.2403395.
- [26] P. J. Huber, "Robust estimation of a location parameter," *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, Mar. 1964, doi: 10.1214/aoms/1177703732.

- [27] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, San Diego, CA, USA, May 2015. [Online]. Available: <https://arxiv.org/abs/1412.6980>
- [28] Z. Zhang, S. Gong, Z. Liu, and D. Chen, "A novel hybrid framework based on temporal convolution network and transformer for network traffic prediction," *PLOS ONE*, vol. 18, no. 9, art. no. e0288935, Sep. 2023, doi: 10.1371/journal.pone.0288935.
- [29] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are Transformers effective for time series forecasting?," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 37, no. 9, 2023, pp. 11121–11128, doi: 10.1609/aaai.v37i9.26317.

Software

- [30] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, Sep. 2020, doi: 10.1038/s41586-020-2649-2.
- [31] S. Seabold and J. Perktold, "statsmodels: Econometric and statistical modeling with Python," in *Proc. 9th Python in Science Conf. (SciPy)*, Austin, TX, USA, 2010, pp. 92–96, doi: 10.25080/Majora-92bf1922-011.
- [32] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, Vancouver, BC, Canada, Dec. 2019. [Online]. Available: <https://arxiv.org/abs/1912.01703>

Project artefacts

- [33] Source code repository: <https://github.com/Emmanuel-NS/Time-series-forecasting-FA>
- [34] Video presentation: *[insert video URL before submission]*